



(/rol/app/)

Help

Korean

검색

Seunghe...

Sites

홈 (/rol/app/) (home) > Managing Traffic with OpenShift Service Mesh > Chapter 1: Managing Traffic with OpenShift Service Mesh

Managing Traffic with OpenShift Service Mesh

Lesson

Lab Environment

Guided Exercise: Resilience with OpenShift Service Mesh

Configure timeout and retry policies to handle transient failures and slow service responses.

Implement connection pooling to limit concurrent connections and protect services from overload.

Outcomes

- Add fault injection with timeout and retry settings to prove the resilience techniques.
- Configure connection pooling to protect services from overload.
- Configure circuit breakers to manage unhealthy service instances.

As the student user on the workstation machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start meshtraffic-resilience
```

Also, run the following command to update your `$PATH` variable and make the `traffic_gen.py` command available immediately. Run it once after creating a new environment.

```
[student@workstation ~]$ source ~/.bashrc
```

The `lab start` command performs the following activities:

- Creates the `meshtraffic-resilience` namespace.
- Adds the `meshtraffic-resilience` namespace to the service mesh.
- Deploys the ratings and three versions of the reviews application in the `meshtraffic-resilience` namespace.
- Configures the `traffic_gen.py` script to generate traffic.
- Creates initial gateway and virtual service OSSM resources for the reviews service.

Instructions

1. Verify the initial status of the service mesh.

1.1. In a new terminal window, log in to the OpenShift cluster as the developer user with the developer password, then switch to the `meshtraffic-resilience` project:

```
[student@workstation ~]$ oc login -u developer \
-p developer https://api.ocp4.example.com:6443
Login successful.
...output omitted...
[student@workstation ~]$ oc project meshtraffic-resilience
...output omitted...
```

1.2. Verify that the applications are running in the `meshtraffic-resilience` namespace.



```
[student@workstation ~]$ oc get pods
NAME                READY   STATUS    RESTARTS   AGE
ratings-v1-...      2/2     Running   0           5m
reviews-v1-...       2/2     Running   0           5m
reviews-v2-...       2/2     Running   0           5m
reviews-v3-...       2/2     Running   0           5m
```

All pods should show 2/2 containers ready, indicating that Istio injected the Envoy sidecar proxy.

1.3. Change to the exercise directory.

```
[student@workstation ~]$ cd ~/course/labs/meshtraffic-resilience
[student@workstation meshtraffic-resilience]$
```

1.4. Verify that the required gateway and virtual service OSSM resources exist in the project.

```
[student@workstation meshtraffic-resilience]$ oc get \
gateways.networking.istio.io,\
virtualservices.networking.istio.io
NAME                                     AGE
gateway.networking.istio.io/reviews-gateway  36m

NAME                                     GATEWAYS          HOSTS          AGE
virtualservice.networking.istio.io/reviews-vs  ["reviews-gateway"]  ["*"]  36m.
```

1.5. Verify correct external access to the mesh by using the traffic_gen.py script.

```
[student@workstation meshtraffic-resilience]$ traffic_gen.py finite.yaml
🚀 Finite mode: 10 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
🔗 curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/10] ✅ HTTP 200 -- No stars (11.0ms)
[2/10] ✅ HTTP 200 -- black (22.8ms)
[3/10] ✅ HTTP 200 -- No stars (9.5ms)
[4/10] ✅ HTTP 200 -- red (22.4ms)
[5/10] ✅ HTTP 200 -- black (20.4ms)
```

...output omitted...

📊 Traffic Statistics

Total Request	Success Rate	Average	P50	P95
10	100.0% (10/10)	16.5ms	17.4ms	22.8ms

📈 Response Distribution

Response	Count	Percentage
black	4	40.0%
No stars	3	30.0%
red	3	30.0%

Your values might differ.

Notice that all requests succeed with balanced traffic distribution. The baseline average response time is approximately 17ms. This provides a reference for observing the impact of resilience configurations in subsequent steps.

NOTE

Remember to run the `source ~/.bashrc` command to update your `$PATH` variable and make the `traffic_gen.py` script available.

```
[student@workstation ~]$ source ~/.bashrc
```

2. Test time-out behavior by injecting delay faults.

In this step, you inject a 2-second delay into 50% of requests to the ratings service. This simulates network latency or slow service responses.

Testing delays enables you to do the following:

- Verify that timeout configurations work correctly
- Observe how delays propagate through service dependencies
- Test retry mechanisms under latency conditions
- Identify services sensitive to downstream latency

2.1. Review the ratings-vs-delay.yaml file that creates a virtual service for the ratings service with a fixed delay.

- Configures fault injection with a delay.
- Applies a 2-second delay to 50% of requests.

```
[student@workstation meshtraffic-resilience]$ cat ratings-vs-delay.yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: ratings-vs
spec:
  hosts:
  - ratings.meshtraffic-resilience.svc.cluster.local
  http:
  - match:
    - uri:
        prefix: /ratings
    fault: ❶
      delay: ❷
        percentage:
          value: 50 ❸
        fixedDelay: 2s ❹
    route:
    - destination:
        host: ratings
        port:
          number: 9080
```

- ❶ Configures fault injection for testing
- ❷ Specifies delay injection
- ❸ Percentage of requests receiving the delay
- ❹ Duration of the delay

2.2. Create the ratings-vs virtual service OSSM resource.

```
[student@workstation meshtraffic-resilience]$ oc create -f ratings-vs-delay.yaml
virtualservice.networking.istio.io/ratings-vs created
```

2.3. Use the traffic_gen.py script to verify that the delay is applied to all requests.

```
[student@workstation meshtraffic-resilience]$ traffic_gen.py finite.yaml
Finite mode: 10 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/10] ✓ HTTP 200 -- black (28.8ms)
[2/10] ✓ HTTP 200 -- red (18.0ms)
[3/10] ✓ HTTP 200 -- No stars (15.9ms)
[4/10] ✓ HTTP 200 -- black (19.6ms)
[5/10] ✓ HTTP 200 -- black (2017.1ms)
[6/10] ✓ HTTP 200 -- black (24.9ms)
[7/10] ✓ HTTP 200 -- black (2016.4ms)
[8/10] ✓ HTTP 200 -- No stars (10.5ms)
[9/10] ✓ HTTP 200 -- red (17.0ms)
[10/10] ✓ HTTP 200 -- red (2026.8ms)

Traffic Statistics
=====
```

Total Request	Success Rate	Average	P50	P95
10	100.0% (10/10)	619.5ms	24.9ms	2026.8ms

```
...output omitted...
```

Notice how approximately 50% of requests to the reviews service versions that depend on ratings (v2/black and v3/red) now show two additional seconds of delay.

3. Add a timeout of one second to the review-vs virtual service.

In this step, you configure a one-second timeout for the reviews-vs virtual service. When the reviews service calls the ratings service and the response exceeds one second, the request fails with an HTTP 504 (Gateway Timeout) error.

This timeout prevents the reviews service from waiting for the full two-second ratings delay, protecting the system by releasing resources and preventing slow responses from cascading to clients.

3.1. Review the reviews-vs-timeout.yaml file that adds a timeout of one second to the reviews-vs virtual service.

- Adds a one-second timeout for all requests.

```
[student@workstation meshtraffic-resilience]$ cat reviews-vs-timeout.yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews-vs
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match:
    - uri:
        prefix: /reviews
    route:
    - destination:
        host: reviews
        port:
          number: 9080
    timeout: 1s ❶
```

- ❶ Configures a one-second timeout for all requests to the reviews service.

3.2. Update the reviews-vs virtual service OSSM resource.

```
[student@workstation meshtraffic-resilience]$ oc replace -f reviews-vs-timeout.yaml
virtualservice.networking.istio.io/reviews-vs replaced
```

3.3. Re-generate the traffic to verify that some requests to the ratings service fail due to exceeding the allowed one second timeout.

```
[student@workstation meshtraffic-resilience]$ traffic_gen.py finite.yaml
Finite mode: 10 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/10] ✓ HTTP 200 -- No stars (86.4ms)
[2/10] ✓ HTTP 200 -- red (98.1ms)
[3/10] ✓ HTTP 200 -- red (17.3ms)
[4/10] ✓ HTTP 200 -- black (15.8ms)
[5/10] ✗ HTTP 504 -- (bad-json) (1004.3ms)
[6/10] ✗ HTTP 504 -- (bad-json) (1005.0ms)
[7/10] ✗ HTTP 504 -- (bad-json) (1003.3ms)
[8/10] ✓ HTTP 200 -- No stars (8.1ms)
[9/10] ✓ HTTP 200 -- red (16.1ms)
[10/10] ✗ HTTP 504 -- (bad-json) (1003.3ms)

Traffic Statistics
=====
```

Total Request	Success Rate	Average	P50	P95
10	60.0% (6/10)	425.8ms	98.1ms	1005.0ms

```
...output omitted...
```

Your values might differ.

Notice that requests exceeding the one-second timeout fail with HTTP 504 errors, whereas requests that complete within the timeout succeed. The timeout protects the system by preventing requests from waiting indefinitely for slow dependencies.

4. Add a Retry policy to the reviews-vs virtual service.

In this step, you configure automatic retries to handle transient failures. The retry policy attempts to recover from temporary errors by retrying failed requests.

This configuration enables you to:

- Recover automatically from transient network failures
- Handle temporary service overloads without manual intervention
- Improve service availability without modifying application code
- Maintain user experience during brief service disruptions

4.1. Review the reviews-vs-retries.yaml file that adds the retries to the reviews-vs virtual service.

- Configures 10 retry attempts for failed requests.
- Sets a 100 millisecond timeout for each retry attempt.
- Retries requests that fail with HTTP 5xx status codes.

```
[student@workstation {gls_lab_script}]$ cat reviews-vs-retries.yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews-vs
spec:
  gateways:
  - reviews-gateway
  hosts:
  - '*'
  http:
  - match:
    - uri:
        prefix: /reviews
    retries: ①
      attempts: 10 ②
      perTryTimeout: 0.1s ③
      retryOn: 5xx ④
    route:
    - destination:
        host: reviews
        port:
          number: 9080
      timeout: 1s
```

- ① Configures retry behavior for the virtual service.
- ② Number of retry attempts (total requests: 1 initial + 10 retries).
- ③ Timeout for each individual retry attempt.
- ④ Retry condition: retry all requests that fail with 5xx HTTP status codes.

4.2. Update the reviews-vs virtual service OSSM resource.

```
[student@workstation meshtraffic-resilience]$ oc replace -f reviews-vs-retries.yaml
virtualservice.networking.istio.io/reviews-vs replaced
```

4.3. Verify that all the requests now succeed due to the Retry policy.

```
[student@workstation meshtraffic-resilience]$ traffic_gen.py finite.yaml
Finite mode: 10 requests to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1/10] ✓ HTTP 200 -- red (49.4ms)
[2/10] ✓ HTTP 200 -- No stars (85.5ms)
[3/10] ✓ HTTP 200 -- No stars (8.1ms)
[4/10] ✓ HTTP 200 -- red (19.0ms)
[5/10] ✓ HTTP 200 -- black (46.5ms)
[6/10] ✓ HTTP 200 -- red (14.7ms)
[7/10] ✓ HTTP 200 -- red (161.3ms)
[8/10] ✓ HTTP 200 -- black (19.7ms)
[9/10] ✓ HTTP 200 -- No stars (7.6ms)
[10/10] ✓ HTTP 200 -- No stars (131.9ms)
```

 Traffic Statistics

```
=====
```

Total Request	Success Rate	Average	P50	P95
10	100.0% (10/10)	54.4ms	46.5ms	161.3ms

...output omitted...

Notice that all requests now succeed with a 100% success rate. The retry policy automatically recovers from transient timeout failures, demonstrating how retries improve service availability without application code changes.

5. Add the Circuit Breaker functionality to the reviews service.

In this step, you configure circuit breakers to protect the reviews service from overload. Circuit breakers limit concurrent connections and pending requests to prevent service degradation.

Testing circuit breakers enables you to do the following:

- Verify that connection pooling limits protect services from overload
- Observe how traffic is rejected when limits are exceeded
- Test system behavior under high concurrency
- Validate that connection limits prevent cascading failures

5.1. Review the contents of the `reviews-dr.yaml` file to check the traffic policy that applies the Circuit Breaker.

- Limits TCP connections to one per replica
- Limits HTTP pending requests to one per replica
- Limits requests per connection to one

```
[student@workstation meshtraffic-resilience]$ cat reviews-dr.yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: reviews-dr
spec:
  host: reviews
  trafficPolicy: ❶
    connectionPool: ❷
      tcp:
        maxConnections: 1 ❸
      http:
        http1MaxPendingRequests: 1 ❹
        maxRequestsPerConnection: 1 ❺
```

- ❶ Traffic policy defining circuit breaker behavior
- ❷ Connection pool settings to limit concurrent connections
- ❸ Maximum of one simultaneous TCP connection per replica
- ❹ Maximum of one pending HTTP request per replica
- ❺ Maximum of one request per connection before closing it

5.2. Create the `reviews-dr` destination rule OSSM resource.

```
[student@workstation meshtraffic-resilience]$ oc create -f reviews-dr.yaml
destinationrule.networking.istio.io/reviews-dr created
```

5.3. Verify that the `reviews` service can only respond to a minority of requests simultaneously by generating traffic in parallel.

```
[student@workstation meshtraffic-resilience]$ traffic_gen.py parallel.yaml
🔄 Continuous mode: 5s to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
🔗 curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1] ✅ HTTP 200 -- red (444.3ms)
[2] ❌ HTTP 503 -- (bad-json) (384.8ms)
[3] ✅ HTTP 200 -- black (625.5ms)
[4] ❌ HTTP 503 -- (bad-json) (383.8ms)
[5] ❌ HTTP 504 -- (bad-json) (1315.1ms)
[6] ✅ HTTP 200 -- red (425.2ms)
[7] ❌ HTTP 503 -- (bad-json) (380.9ms)
[8] ✅ HTTP 200 -- black (815.5ms)
[9] ✅ HTTP 200 -- black (377.4ms)
[10] ✅ HTTP 200 -- red (1042.7ms)
...output omitted...

📊 Traffic Statistics
=====
```

Total Request	Success Rate	Average	P50	P95
160	29.4% (47/160)	59.9ms	4.3ms	383.8ms

```
...output omitted...
```

Your values might differ.

Notice the low 29.4% success rate when the connection limit is set to one. The circuit breaker rejects most concurrent requests with HTTP 503 errors, protecting the service from overload by enforcing strict connection limits.

5.4. Review the `reviews-dr-2.yaml` file to verify that it changes the number of concurrent request allowed to the reviews service.

- Increases TCP connections to four per replica.
- Increases HTTP pending requests to four per replica.
- Increases requests per connection to four.

```
[student@workstation meshtraffic-resilience]$ cat reviews-dr-2.yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: reviews-dr
spec:
  host: reviews
  trafficPolicy:
    connectionPool:
      tcp:
        maxConnections: 4 ❶
      http:
        http1MaxPendingRequests: 4 ❷
        maxRequestsPerConnection: 4 ❸
```

- ❶ Increased to four simultaneous TCP connections per replica.
- ❷ Increased to four pending HTTP requests per replica.
- ❸ Increased to four requests per connection.

5.5. Update the `reviews-dr` destination OSSM resource.

```
[student@workstation meshtraffic-resilience]$ oc replace -f reviews-dr-2.yaml
destinationrule.networking.istio.io/reviews-dr replaced
```

5.6. Verify that the reviews service responds to more requests simultaneously when generating traffic in parallel.

```
[student@workstation meshtraffic-resilience]$ traffic_gen.py parallel.yaml
🔄 Continuous mode: 5s to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
🔗 curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
[1] ✅ HTTP 200 -- red (31.3ms)
[2] ✅ HTTP 200 -- black (88.9ms)
[3] ✅ HTTP 200 -- red (28.6ms)
[4] ✅ HTTP 200 -- No stars (122.7ms)
[5] ✅ HTTP 200 -- No stars (15.8ms)
[6] ✅ HTTP 200 -- red (26.6ms)
[7] ❌ HTTP 503 -- (bad-json) (3.0ms)
[8] ✅ HTTP 200 -- red (126.0ms)
[9] ✅ HTTP 200 -- red (26.4ms)
[10] ❌ HTTP 503 -- (bad-json) (2.2ms)
...output omitted...

📊 Traffic Statistics
=====
```

Total Request	Success Rate	Average	P50	P95
120	95.0% (114/120)	130.1ms	119.4ms	334.3ms

```
...output omitted...
```

Your values might differ.

Notice the improved 95.0% success rate when the connection limit is increased to four. By increasing the connection pool capacity, the circuit breaker can handle more concurrent requests, demonstrating how connection pooling limits directly impact service availability under load.

Finish

On the workstation machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish meshtraffic-resilience
```

Revision: ad0008I-3.1-ed4-20260310-8ae9760



개인 정보 취급 방침 (https://www.redhat.com/ko/about/privacy-policy?extIdCarryOver=true&sc_cid=701f2000001D8QoAAK)

Red Hat 교육 정책 (<https://www.redhat.com/ko/about/red-hat-training-policies>)

이용 약관 (<https://www.redhat.com/ko/about/terms-use>)

모든 정책 및 지침 (<https://www.redhat.com/ko/about/all-policies-guidelines>)

릴리스 노트
(https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

쿠키 기본설정